Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

15
서브워드 토크나이저

세상의 모든 단어를 단어 집합에 넣을 수는 없습니다. 모르는 단어를 <UNK>로 버리는 대신, 아는 조각들로 쪼개어 읽는 방법을 배웁니다. BERT와 GPT가 글을 읽는 첫 단계가 바로 이것입니다.

오늘 다루는 것 (15-01 바이트 페어 인코딩)

1. OOV 문제와 서브워드, BPE로 단어 집합 만들기
2. 새 단어 쪼개기, WordPiece, 실제 토크나이저

끝나고 나면 할 수 있는 것

1. OOV 문제가 무엇이고 서브워드가 왜 해결책이 되는지 설명할 수 있다.
2. 단어 4개짜리 예에서 BPE가 합치는 순서를 손으로 따라갈 수 있다.
3. 처음 보는 단어가 학습된 규칙으로 어떻게 쪼개지는지 설명할 수 있다.

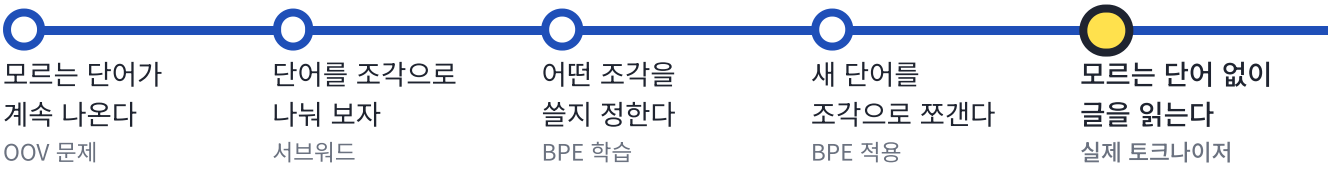
2026년 10월 13일 화요일 14:30 (10/13 자료 3/3)
실습 1, 2는 설치 없이 실행됩니다. 실습 3만 코랩에서 `!pip install tokenizers gensim` 이 필요합니다.

15

서브워드 토크나이저

13장에서 단어 집합을 만들 때, 드물게 나오는 단어는 버리고 처음 보는 단어는 모두 <UNK>로 바꿨습니다. 그만큼의 정보를 잃은 것입니다. 서브워드 토크나이저는 단어를 더 작은 조각(서브워드)으로 나눠, 한정된 크기의 단어 집합으로도 어떤 단어든 표현할 수 있게 합니다.

왜 배우나: 오늘의 학습 경로



교재의 15-01 한 소챕터를 두 부분으로 나눴습니다. 예시는 교재와 같이 단어 네 개(low, lower, newest, widest)로 된 작은 훈련 데이터입니다. 여기에 없던 단어 **lowest**를 읽을 수 있게 되는 과정을 끝까지 따라갑니다.

15-01 ① OOV 문제와 BPE

핵심 한 줄

BPE는 글자 단위에서 시작해, 가장 자주 이웃해 나오는 두 조각을 하나로 합치는 일을 정해진 횟수만큼 반복해서 서브워드 단어 집합을 만든다.

왜 배우나

단어 단위로 자르면 단어 집합에 없는 단어는 읽을 수 없습니다. 글자 단위로 자르면 모르는 것은 없어지지만 글자 하나에는 뜻이 거의 없고 문장이 너무 길어집니다. 서브워드는 그 사이입니다. 문제는 "어떤 조각을 단어 집합에 넣을 것인가"이고, BPE가 그 답을 데이터에서 찾아 줍니다.

1) OOV 문제와 서브워드

정의

- OOV(Out-Of-Vocabulary): 단어 집합에 없는 단어. 모델에 넣을 때는 <UNK>(unknown) 토큰 하나로 바뀝니다. 이 때문에 문제를 풀기 어려워지는 상황을 OOV 문제라고 합니다.
- 서브워드(subword): 단어보다 작은, 뜻을 가질 수 있는 조각. 예: birthplace = birth + place
- 서브워드 분리(subword segmentation): 단어를 서브워드들로 나누는 전처리. 이 일을 하는 도구가 서브워드 토크나이저입니다.

왜 OOV가 계속 생길까? 새 말(신조어), 이름, 드문 단어, 오타가 끝없이 나오기 때문입니다. 단어 집합을 아무리 키워도 다 담을 수 없고, 키울수록 모델도 커집니다.

자르는 단위	단어 집합의 크기	모르는 단어	토큰 하나의 뜻	문장의 길이
단어	매우 큼 (수만~수십만)	많이 생김	분명함	짧음
글자	매우 작음 (수십~수천)	거의 없음	거의 없음	매우 김
서브워드	정할 수 있음 (보통 수천~수만)	거의 없음	어느 정도 있음	조금 길어짐

예시 적용

훈련 데이터에 low(5번), lower(2번), newest(6번), widest(3번)만 있었다고 합시다. 단어 단위 단어 집합은 이 네 단어입니다. 시험 때 **lowest**가 나오면 단어 집합에 없으므로 <UNK>가 됩니다. 하지만 low도 알고 -est로 끝나는 단어도 둘이나 봤습니다. 이 둘로 나눠 읽을 수 있다면 뜻을 짐작할 수 있습니다.



그림 1 단어 단위에서는 lowest가 <UNK>가 되지만, 서브워드 단위에서는 아는 조각 low와 est로 나뉜다

2) 원래의 BPE: 압축 알고리즘

정의

- BPE(Byte Pair Encoding, 바이트 페어 인코딩): 1994년에 나온 데이터 압축 방법. 문자열에서 **가장 자주 이어서 나오는 두 글자의 쌍**을 찾아 새 글자 하나로 바꾸는 일을, 더 바꿀 쌍이 없을 때까지 반복합니다.
- 바꾼 규칙(예: Z = aa)을 적어 두면, 거꾸로 적용해 원래 문자열을 되살릴 수 있습니다.

예시 적용

문자열 `aaabdaaabac` (11글자)를 압축합니다(실습 1).

단계	가장 자주 나온 쌍	바꾼 뒤	길이
처음		aaabdaaabac	11
1	aa (4번) → Z	ZabdZabac	9
2	Za (2번) → Y	YbdYbac	7
3	Yb (2번) → X	XdXac	5

2단계에서 교재는 ab를 골랐고 실습은 Za를 골랐습니다. 둘 다 2번씩 나와 횟수가 같기 때문이고, 어느 쪽을 골라도 마지막 결과는 XdXac로 같습니다. 규칙을 거꾸로 적용하면 원래 문자열이 됩니다(True).

3) 자연어 처리의 BPE: 서브워드 단어 집합 만들기

정의

2016년에 이 압축 방법을 단어를 나누는 데 쓰자는 논문이 나왔습니다. "자주 붙어 다니는 글자 묶음은 하나의 조각으로 볼 만하다"는 생각입니다. 글자에서 시작해 위로 쌓아 가는 방식(bottom-up)입니다.

1. 훈련 데이터의 단어마다 등장 횟수를 셉니다(이 표를 교재는 딕셔너리라고 부릅니다).

2. 모든 단어를 글자 단위로 쪼갭니다. 단어의 끝에는 끝 표시 `</w>` 를 붙입니다. 처음의 단어 집합은 글자들입니다.
3. 이웃한 두 조각의 쌍마다, 그 쌍이 나온 횟수를 셉니다. 단어가 5번 나왔다면 그 안의 쌍도 5번으로 셉니다.
4. 횟수가 가장 큰 쌍을 하나로 합치고, 합친 조각을 단어 집합에 넣습니다.
5. 3~4를 사용자가 정한 횟수(`num_merges`)만큼 반복합니다.

- **합치는 횟수가 곧 단어 집합의 크기를 정합니다.** 단어 집합의 크기 = 처음의 글자 수 + 합친 횟수. 많이 합칠수록 조각이 길어져 단어에 가까워지고, 적게 합칠수록 글자에 가까워집니다.
- **합친 순서를 기록해 둡니다.** 새 단어를 쪼갤 때 이 순서를 그대로 씁니다(②부).
- **`</w>` 는 왜 붙일까?** 단어의 끝에 오는 est(newest의 est)와 중간에 오는 est(estimate의 est)를 구분하고, 조각들을 다시 붙일 때 어디서 띄어 써야 하는지 알기 위해서입니다.

예시 적용

low(5), lower(2), newest(6), widest(3)에 10번 합치기를 적용합니다(실습 2).



그림 2 BPE가 조각을 합쳐 가는 과정. 자주 나오는 단어 newest는 통째로 하나가 되고, 드문 lower는 low + e + r로 남는다

- **1회**: (e, s)가 9번으로 가장 많습니다. newest에서 6번, widest에서 3번입니다. es로 합칩니다.
- **2~3회**: (es, t)도 9번, (est, </w>)도 9번입니다. "단어 끝의 est"가 한 조각이 됩니다.
- **4~5회**: (l, o), (lo, w)가 7번(low 5 + lower 2)입니다. low가 한 조각이 됩니다.
- **6~8회**: n, e, w, est가 차례로 합쳐져 newest가 통째로 한 조각이 됩니다(6번).
- **10회 뒤**: 합친 순서는 es, est, est</w>, lo, low, ne, new, newest</w>, low</w>, wi 입니다.

결과를 보면 "est로 끝난다", "low로 시작한다"처럼 사람이 보기에도 뜻이 있는 조각이 저절로 나왔습니다. 문법을 알려 준 적은 없고 횟수만 셌습니다.

비유

레고 블록으로 단어 만들기

완성된 모형(단어)만 상자에 넣어 두면, 상자에 없는 모형은 만들 수 없습니다. 가장 작은 블록(글자)만 넣어 두면 무엇이든 만들 수 있지만 매번 처음부터 조립해야 합니다. 그래서 자주 함께 쓰는 블록 묶음(바퀴 세트, 창문 세트)을 미리 조립해 넣어 둡니다. 어떤 묶음을 만들어 둘지는 "지금까지 얼마나 자주 함께 썼는지"를 보고 정합니다.

완성된 모형	= = 단어
가장 작은 블록	= = 글자
미리 조립한 묶음	= = 서브워드
자주 함께 쓴 블록부터 묶기	= = BPE의 합치기

실습 1

원래의 BPE: 문자열 압축하기

```python
from collections import Counter

# 원래의 BPE(1994): 가장 자주 나오는 글자 쌍을 새 글자 하나로 바꾸는 압축 방법
text = "aaabdaaabac"
new_symbols = iter("ZYXWV")
rules = {}
print("처음:", text, f"({len(text)}글자)")

while True:
    pairs = Counter(text[i:i + 2] for i in range(len(text) - 1))
    pair, count = pairs.most_common(1)[0]
    if count < 2:                       # 두 번 이상 나오는 쌍이 없으면 끝
        break
    symbol = next(new_symbols)
    rules[symbol] = pair
    text = text.replace(pair, symbol)
    print(f"'{pair}'가 {count}번 → '{symbol}'로 바꿈: {text}",
          f"({len(text)}글자)")

print("바꾼 규칙:", rules)

# 규칙을 거꾸로 적용하면 원래 문자열로 돌아온다
restored = text
for symbol, pair in reversed(rules.items()):
    restored = restored.replace(symbol, pair)
print("복원:", restored, " 원래와 같은가?", restored == "aaabdaaabac")
```

실행 결과

```
처음: aaabdaaabac (11글자)
'aa'가 4번 → 'Z'로 바꿈: ZabdZabac (9글자)
'Za'가 2번 → 'Y'로 바꿈: YbdYbac (7글자)
'Yb'가 2번 → 'X'로 바꿈: XdXac (5글자)
바꾼 규칙: {'Z': 'aa', 'Y': 'Za', 'X': 'Yb'}
복원: aaabdaaabac  원래와 같은가? True
```

직접 해 보기 `text`를 `"abababcabab"`로 바꾸면 어떤 쌍부터 합쳐지나요? 몇 글자까지 줄어드나요?

실습 2 BPE로 서브워드 단어 집합을 만들고, 새 단어 쪼개기

```python
from collections import Counter

# 훈련 데이터의 단어와 등장 횟수. 단어를 글자로 쪼개고 끝에 </w>를 붙인다
dictionary = {("l", "o", "w", "</w>"): 5,
              ("l", "o", "w", "e", "r", "</w>"): 2,
              ("n", "e", "w", "e", "s", "t", "</w>"): 6,
              ("w", "i", "d", "e", "s", "t", "</w>"): 3}
num_merges = 10                         # 몇 번 합칠지는 사용자가 정한다

def count_pairs(dictionary):
    """이웃한 두 조각이 함께 나온 횟수 (단어의 등장 횟수만큼 센다)"""
    pairs = Counter()
    for symbols, freq in dictionary.items():
        for a, b in zip(symbols, symbols[1:]):
            pairs[a, b] += freq
    return pairs

def merge(symbols, pair):
    """조각의 줄에서 pair가 이어진 곳을 하나로 합친다"""
    out, i = [], 0
    while i < len(symbols):
        if symbols[i:i + 2] == pair:
            out.append(pair[0] + pair[1])
            i += 2
        else:
            out.append(symbols[i])
            i += 1
    return tuple(out)

# 1) 학습: 가장 자주 나온 쌍을 합치는 일을 num_merges번 반복한다
merges = []
for step in range(1, num_merges + 1):
    pairs = count_pairs(dictionary)
    best = max(pairs, key=pairs.get)            # 횟수가 같으면 먼저 나온 쌍
    dictionary = {merge(s, best): f for s, f in dictionary.items()}
    merges.append(best)
    words = " | ".join(" ".join(s) for s in dictionary)
    print(f"{step:2d}회: {best} ({pairs[best]}번) → {words}")

vocab = sorted({s for symbols in dictionary for s in symbols})
print("합친 순서:", ["".join(m) for m in merges])

# 2) 새 단어를 쪼개기: 글자로 나눈 뒤, 학습 때 합친 순서대로 다시 합친다
def encode(word):
    symbols = tuple(word) + ("</w>",)
    for pair in merges:
        symbols = merge(symbols, pair)
    return [s.replace("</w>", "") for s in symbols if s != "</w>"]

print()
for word in ["lowest", "newer", "loki", "lowing", "highing"]:
    print(f"{word:8s} → {encode(word)}")
```

실행 결과

```
 1회: ('e', 's') (9번) → l o w </w> | l o w e r </w> | n e w es t </w> | w i d es t </w>
 2회: ('es', 't') (9번) → l o w </w> | l o w e r </w> | n e w est </w> | w i d est </w>
 3회: ('est', '</w>') (9번) → l o w </w> | l o w e r </w> | n e w est</w> | w i d est</w>
 4회: ('l', 'o') (7번) → lo w </w> | lo w e r </w> | n e w est</w> | w i d est</w>
 5회: ('lo', 'w') (7번) → low </w> | low e r </w> | n e w est</w> | w i d est</w>
 6회: ('n', 'e') (6번) → low </w> | low e r </w> | ne w est</w> | w i d
```

```
est</w>
 7회: ('ne', 'w') (6번) → low </w> | low e r </w> | new est</w> | w i d
est</w>
 8회: ('new', 'est</w>') (6번) → low </w> | low e r </w> | newest</w> | w i d
est</w>
 9회: ('low', '</w>') (5번) → low</w> | low e r </w> | newest</w> | w i d
est</w>
10회: ('w', 'i') (3번) → low</w> | low e r </w> | newest</w> | wi d est</w>
합친 순서: ['es', 'est', 'est</w>', 'lo', 'low', 'ne', 'new', 'newest</w>',
'low</w>', 'wi']

lowest   → ['low', 'est']
newer    → ['new', 'e', 'r']
loki     → ['lo', 'k', 'i']
lowing   → ['low', 'i', 'n', 'g']
highing  → ['h', 'i', 'g', 'h', 'i', 'n', 'g']
```

직접 해 보기 `num_merges`를 3으로 줄이면 lowest는 어떻게 쪼개지나요? 13으로 늘리면 widest는 어떻게 되나요?

확인 문제

① 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 단어 집합에 없는 단어를 OOV라고 한다. (O / X) 이름 _____

2 BPE는 가장 드물게 나오는 쌍부터 합친다. (O / X) 이름 _____

3 BPE가 처음에 단어를 쪼개는 단위는? 이름 _____

① 글자 ② 문장 ③ 형태소

4 예시에서 가장 먼저 합쳐진 쌍은? 이름 _____

① (l, o) ② (e, s) ③ (n, e)

5 BPE에서 단어의 끝에 붙이는 표시는 _____이다. 이름 _____

6 단어 집합의 크기는 처음의 글자 수 + _____ 횟수로 정해진다. 이름 _____

7 서브워드가 단어 단위와 글자 단위의 문제를 어떻게 함께 줄이는지, 레고 블록에 빗대어 설명해 보세요. 이름 _____

15-01 ② 새 단어 쪼개기와 실제 토크나이저

핵심 한 줄

처음 보는 단어는 글자로 나눈 뒤 학습 때 합친 순서대로 다시 합쳐서 쪼개며, 이렇게 하면 같은 크기의 단어 집합으로도 <UNK>가 거의 생기지 않는다.

왜 배우나

단어 집합을 만드는 것은 절반입니다. 나머지 절반은 그 단어 집합으로 새 글을 실제로 자르는 일입니다. 그리고 실무에서는 BPE를 직접 짜지 않고 라이브러리를 씁니다. 다음에 배울 BERT는 WordPiece를, GPT는 BPE를 쓰므로 여기서 한 번 써 봅니다.

1) 새 단어를 서브워드로 쪼개기

정의

1. 단어를 글자로 나누고 끝에 `</w>` 를 붙입니다.
2. 학습 때 기록한 **합친 순서대로**, 그 쌍이 단어 안에 이웃해 있으면 합칩니다.
3. 기록된 쌍을 모두 훑고 나서 남은 조각들이 결과입니다.

- 학습 때 본 적 있는 묶음은 조각으로 살아나고, 본 적 없는 부분은 글자로 남습니다.
- 글자는 모두 단어 집합에 있으므로, 어떤 단어도 최악의 경우 글자들로 표현됩니다. 그래서 <UNK>가 생기지 않습니다(훈련 데이터에 한 번도 없던 글자만 예외입니다).

예시 적용

①부에서 학습한 합친 순서로 새 단어 다섯 개를 쪼갭니다(실습 2). lowest는 이렇게 진행됩니다.

`l o w e s t </w>` → `l o w es t </w>` → `l o w est </w>` → `l o w est</w>` → `lo w est</w>` → `low est</w>`

새 단어	쪼갠 결과	읽는 법
lowest	low + est	둘 다 학습 때 만든 조각. 훈련 데이터에 없던 단어를 아는 조각 둘로 읽는다
newer	new + e + r	new는 살아나고, er은 합친 적이 없어 글자로 남는다
loki	lo + k + i	lo만 아는 조각
lowing	low + i + n + g	ing를 합친 적이 없어 글자로 남는다
highing	h + i + g + h + i + n + g	아는 묶음이 하나도 없어 모두 글자

한계도 보입니다. 훈련 데이터가 단어 네 개뿐이라 ing, er 같은 흔한 조각을 배우지 못했습니다. 조각의 질은 훈련 데이터에 달려 있습니다. 다음 절에서 큰 데이터로 다시 해 봅니다.

2) WordPiece, Unigram, 그리고 실제 토크나이저

정의

BPE를 바탕으로 한 다른 방법이 둘 있습니다. 세 방법 모두 "서브워드 단어 집합을 만든다"는 목표는 같고, 어떤 조각을 고르는 기준이 다릅니다.

방법	단어 집합을 만드는 기준	쓰는 모델
BPE	가장 **자주** 이웃한 쌍을 합친다	GPT 계열
WordPiece	합쳤을 때 말뭉치의 **우도**를 가장 높이는 쌍을 합친다	BERT
Unigram	큰 후보 집합에서 시작해, 없애도 우도가 가장 덜 떨어지는 조각을 **없애 나간다**	T5 등 (SentencePiece)

- 우도(likelihood): 지금의 단어 집합으로 훈련 글이 나올 확률(10장의 언어 모델). WordPiece는 "따로 있을 때보다 붙어 있을 때가 얼마나 더 자주인가"를 봅니다. 단순히 많이 나온 쌍이 아니라 서로 유난히 잘 붙어 다니는 쌍을 고릅니다.
- **조각의 경계를 표시하는 방법**도 다릅니다. BPE 논문은 단어 끝에 `</w>` 를, WordPiece(BERT)는 단어 중간에 오는 조각 앞에 `##` 을, SentencePiece는 단어의 맨 앞에 `▁` 를 붙입니다. 어느 방식이든 "조각을 붙여 원래 띄어쓰기를 되살릴 수 있게" 하려는 것입니다.
- 실무에서는 SentencePiece(구글)나 허깅페이스의 tokenizers 라이브러리를 씁니다. 훈련 글과 원하는 단어 집합의 크기를 주면 학습해 줍니다.

예시 적용

위키백과 발췌본 text8의 앞 300만 단어로 학습하고, 그 뒤 50만 단어로 시험했습니다(실습 3). 학습 글에 나온 서로 다른 단어는 96,644개입니다.

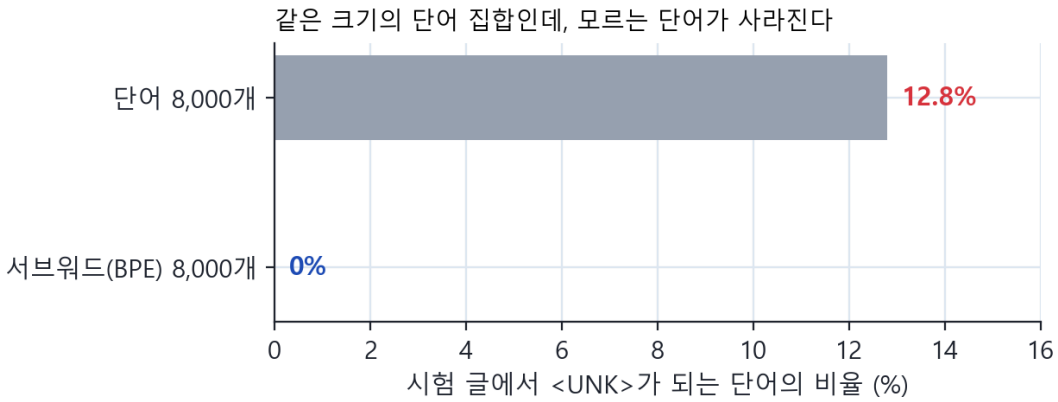


그림 3 단어 집합의 크기를 똑같이 8,000으로 두었을 때, 시험 글에서 <UNK>가 되는 단어의 비율(실습 3)

- **단어 단위**: 자주 나온 단어 8,000개만 알면 시험 글의 12.8%가 <UNK>입니다. 여덟 단어 중 하나를 읽지 못합니다.
- **BPE**: 서브워드 8,000개로는 <UNK>가 0개입니다. 대신 단어 하나가 평균 1.24조각이 되어 문장이 24% 길어집니다.

- 자주 나오는 단어는 통째로 남습니다: the first world war → [the, first, world, war]

처음 보는 단어	단어 단위	BPE
unhappiness	<UNK>	un + happ + iness
transformers	<UNK>	trans + form + ers
koreans	<UNK>	kore + ans
chatgpt	<UNK>	chat + g + pt
electrofishing	<UNK>	electr + of + ishing

- **잘 된 경우**: un-, -iness, trans-, -ers처럼 뜻이 있는 조각으로 나뉩니다.
- **이상한 경우**: electrofishing은 electro + fishing이 아니라 electr + of + ishing으로 나뉘었습니다. BPE는 뜻을 보지 않고 횟수만 보기 때문입니다. of가 워낙 자주 나오는 조각이라 먼저 합쳐졌습니다.
- **WordPiece**는 같은 글로 학습해도 다르게 나눕니다: unhappiness → un + ##ha + ##pp + ##iness, transformers → transform + ##ers. ##은 "앞 조각에 붙는다"는 표시입니다.
- **되돌리기**: BPE로 자른 "unhappiness is temporary"를 번호에서 되돌리면 "un happ iness is temporary"가 됩니다. 이 실습의 BPE에는 끝 표시가 없어 어디를 붙여야 할지 모르기 때문입니다. `</w>`나 `##`이 필요한 이유입니다.

비유

처음 보는 한자어 읽기

"초전도체"라는 말을 처음 봐도, "초(넘다)", "전도(전기를 전하다)", "체(물체)"를 알면 뜻을 짐작할 수 있습니다. 통째로 외운 적은 없지만 아는 조각으로 나눠 읽은 것입니다. 아는 조각이 하나도 없는 말은 한 글자씩 읽는 수밖에 없습니다.

처음 보는 낱말 = = OOV 단어
아는 한자 조각 = = 서브워드
조각으로 나눠 읽기 = = 서브워드 분리
한 글자씩 읽기 = = 글자 단위로 남는 경우

실습 3

tokenizers 라이브러리로 BPE와 WordPiece 학습하기

```
# 코랩에서는 먼저:  !pip install tokenizers gensim   (약 1분)
import gzip
from collections import Counter

import gensim.downloader as api
from tokenizers import Tokenizer
from tokenizers.models import BPE, WordPiece
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.trainers import BpeTrainer, WordPieceTrainer
```

```python
# 1) 데이터: 위키백과 발췌본 text8. 앞 300만 단어로 학습, 그 뒤 50만 단어로 시험
path = api.load("text8", return_path=True)
words = gzip.open(path, "rt", encoding="utf8").read().split()
train_words, test_words = words[:3_000_000], words[3_000_000:3_500_000]
lines = [" ".join(train_words[i:i + 1000])
         for i in range(0, len(train_words), 1000)]

# 2) 단어 단위 단어 집합: 자주 나온 단어 8,000개만 안다
counts = Counter(train_words)
print("학습 글의 서로 다른 단어:", f"{len(counts):,}개")
word_vocab = {w for w, _ in counts.most_common(8000)}
unknown = sum(w not in word_vocab for w in test_words)
rate = unknown / len(test_words)
print(f"단어 8,000개짜리 단어 집합 → 시험 글의 {rate:.1%}가 <UNK>")

# 3) BPE로 서브워드 8,000개짜리 단어 집합 만들기
bpe = Tokenizer(BPE(unk_token="[UNK]"))
bpe.pre_tokenizer = Whitespace()                    # 먼저 띄어쓰기로 자른다
options = dict(vocab_size=8000, special_tokens=["[UNK]"], show_progress=False)
bpe.train_from_iterator(lines, BpeTrainer(**options))
ids = bpe.encode(" ".join(test_words)).ids
unk = sum(i == bpe.token_to_id("[UNK]") for i in ids)
print(f"서브워드 8,000개짜리 단어 집합 → 시험 글의 <UNK> {unk}개,",
      f"단어 하나가 평균 {len(ids) / len(test_words):.2f}조각")

# 4) 처음 보는 단어를 어떻게 쪼개나
rare = ["unhappiness", "electrofishing", "transformers", "chatgpt", "koreans"]
for word in rare:
    known = "아는 단어" if word in word_vocab else "<UNK>"
    print(f"  {word:15s} 단어 단위: {known:9s} BPE: {bpe.encode(word).tokens}")
print("  자주 나오는 단어는 통째로:", bpe.encode("the first world war").tokens)

# 5) WordPiece: 단어 중간에 오는 조각 앞에 ##을 붙여 구분한다 (BERT가 쓰는 방식)
wp = Tokenizer(WordPiece(unk_token="[UNK]"))
wp.pre_tokenizer = Whitespace()
wp.train_from_iterator(lines, WordPieceTrainer(**options))
for word in ["unhappiness", "transformers"]:
    print(f"  WordPiece {word}: {wp.encode(word).tokens}")

# 6) 번호로 바꾸고 되돌리기
enc = bpe.encode("unhappiness is temporary")
print("토큰:", enc.tokens)
print("번호:", enc.ids, "→ 되돌리기:", bpe.decode(enc.ids))
```

실행 결과

```
학습 글의 서로 다른 단어: 96,644개
단어 8,000개짜리 단어 집합 → 시험 글의 12.8%가 <UNK>
서브워드 8,000개짜리 단어 집합 → 시험 글의 <UNK> 0개, 단어 하나가 평균 1.24조각
  unhappiness     단어 단위: <UNK>     BPE: ['un', 'happ', 'iness']
  electrofishing  단어 단위: <UNK>     BPE: ['electr', 'of', 'ishing']
  transformers    단어 단위: <UNK>     BPE: ['trans', 'form', 'ers']
  chatgpt         단어 단위: <UNK>     BPE: ['chat', 'g', 'pt']
  koreans         단어 단위: <UNK>     BPE: ['kore', 'ans']
  자주 나오는 단어는 통째로: ['the', 'first', 'world', 'war']
  WordPiece unhappiness: ['un', '##ha', '##pp', '##iness']
  WordPiece transformers: ['transform', '##ers']
토큰: ['un', 'happ', 'iness', 'is', 'temporary']
번호: [80, 2384, 1524, 37, 7619] → 되돌리기: un happ iness is temporary
```

직접 해 보기 `vocab_size`를 8000에서 1000으로 줄이면 "단어 하나가 평균 몇 조각"이 어떻게 달라지나요? unhappiness는 어떻게 쪼개지나요?

② 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 BPE를 쓰면 훈련 데이터에 없던 단어도 서브워드로 나눠 표현할 수 있다. (O / X) 이름 _____

2 BPE는 단어의 뜻을 보고 조각을 나눈다. (O / X) 이름 _____

3 새 단어를 쪼갤 때 합치는 순서는? 이름 _____

① 학습 때 합친 순서 ② 알파벳 순서 ③ 무작위

4 BERT가 쓰는 서브워드 방법은? 이름 _____

① BPE ② WordPiece ③ 형태소 분석

5 예시의 단어 집합으로 lowest를 쪼개면 low + _____가 된다. 이름 _____

6 WordPiece에서 단어 중간에 오는 조각 앞에 붙이는 표시는 _____이다. 이름 _____

7 실습에서 단어 단위와 BPE의 <UNK> 비율이 어떻게 달랐는지 말하고, 그 대신 치르는 대가를 설명해 보세요. 이름 _____

원문: 15-01 바이트 페어 인코딩(BPE) — https://wikidocs.net/217494 (교재의 한 소챕터를 두 부분으로 나눴습니다. BPE 코드는 교재의 코드를 같은 결과가 나오게 짧게 다시 썼고, 실습 3과 방법 비교 표, <UNK> 비율 실험은 이 자료에서 추가했습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
OOV	단어 집합에 없는 단어. <UNK>가 된다	lowest
서브워드	단어보다 작은, 뜻을 가질 수 있는 조각	low, est
BPE 학습	가장 자주 이웃한 쌍을 합치기를 반복	(e, s) 9번 → es
BPE 적용	글자로 나눈 뒤 학습 때의 순서로 합침	lowest → low + est
WordPiece	우도를 가장 높이는 쌍을 합침. ## 표시	transform + ##ers
효과	같은 크기의 단어 집합으로 <UNK>가 사라짐	12.8% → 0%

다음

다음 자료 예고: 16장 시퀀스투시퀀스 (10/14)

지금까지는 문장을 넣어 정답 하나(분류)나 단어마다의 태그를 얻었습니다. 다음에는 문장을 넣어 **다른 문장**을 얻습니다. 번역이 대표적입니다. 입력 문장을 읽는 인코더와 출력 문장을 만드는 디코더, 두 개의 RNN을 이어 붙입니다.

정답과 해설

① OOV 문제와 BPE

1. O
2. X — 가장 자주 나오는 쌍부터 합칩니다.
3. ①
4. ② — newest 6번 + widest 3번 = 9번
5. </w>
6. 합친 (병합)
7. 예시 답: 완성된 모형만 있으면 없는 모형을 못 만들고(단어 단위의 OOV), 가장 작은 블록만 있으면 매번 처음부터 조립해야 한다(글자 단위는 뜻이 없고 길다). 자주 함께 쓰는 블록 묶음을 미리 만들어 두면, 흔한 것은 묶음 하나로, 처음 보는 것은 묶음과 블록을 섞어 만들 수 있다.

② 새 단어 쪼개기와 실제 토크나이저

1. O
2. X — 횟수만 봅니다. 그래서 electr + of + ishing 같은 결과도 나옵니다.
3. ①
4. ②
5. est
6. ##
7. 예시 답: 단어 집합의 크기를 8,000으로 같게 두었을 때 단어 단위는 시험 글의 12.8%가 <UNK>였고 BPE는 0개였다. 대신 단어 하나가 평균 1.24조각으로 나뉘어 문장이 길어지고, 뜻과 맞지 않게 쪼개지는 단어도 생긴다.

용어 정리

용어	정의
OOV, <UNK>	단어 집합에 없는 단어 / 그런 단어를 대신하는 토큰
서브워드	단어보다 작은, 뜻을 가질 수 있는 조각
서브워드 토크나이저	단어를 서브워드로 나누는 도구
BPE	가장 자주 이웃한 쌍을 합치기를 반복해 서브워드 단어 집합을 만드는 방법
</w>	단어의 끝을 나타내는 표시
WordPiece	합쳤을 때 우도를 가장 높이는 쌍을 합치는 방법. BERT가 쓴다
Unigram	큰 후보에서 시작해 필요 없는 조각을 없애 나가는 방법
우도	지금의 단어 집합(모델)에서 훈련 글이 나올 확률
SentencePiece, tokenizers	서브워드 토크나이저를 학습하고 쓰는 라이브러리